Local execution evidence

Three-tier BlogReact application and native Jenkins CI

Executed 22 October 2026 | Windows local demonstration | Version 1.0

This report contains genuine screenshots of the running application and Jenkins jobs, supported by exported build logs, scan reports and database verification results. It replaces repository-only screenshots as the primary evidence for the local demonstration.

Verified outcome

Check	Observed result
Application	React frontend, Express API and real isolated MySQL running locally
Jenkins CI	BlogApp-Local-Demonstration build #2: SUCCESS, 34.448 seconds
Tests and build	7 backend tests and 3 frontend tests passed; production frontend build passed
Dependency gate	Trivy 0.76.1: zero HIGH/CRITICAL findings in both lockfiles, including development dependencies
Gate rejection	Separate fixture job: expected FAILURE with two HIGH findings
Persistence	Same post ID 1 and its comment verified after MySQL restart
Backup restoration	Separate restored database matched source counts and proof-post content

Scope of this submission

This is a native local execution supplement. Docker Desktop did not start, so containers and image scans were not executed. The EKS deployment, SonarQube gate, Nexus/ECR publishing, Grafana and Kubernetes policy/rolling-update proof are not complete. Acceptance as a substitute for the original EKS brief requires the evaluator's agreement.

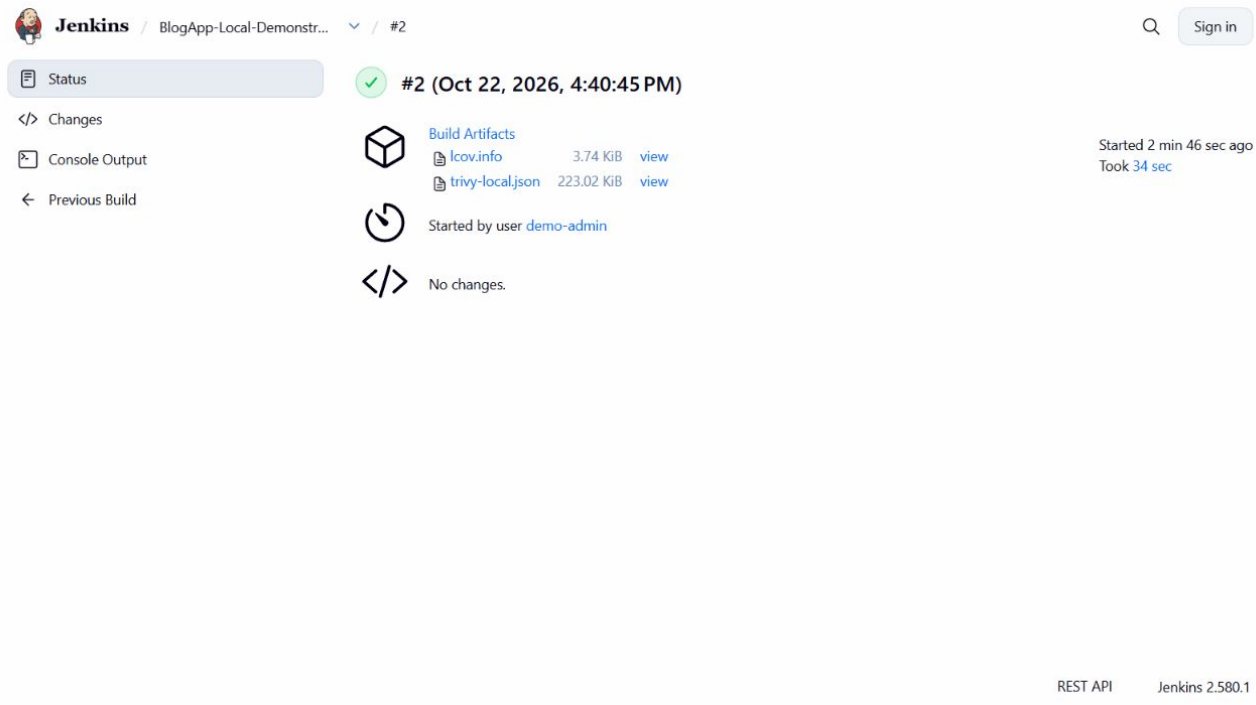
Repository: https://github.com/praveenpreni/BlogReact

Implementation and execution-record revision: 2eddc901ab74f90a6bc7e2239728584fac5e38bf

The report and screenshot publication may use a later documentation-only commit. Executed application source hashes are retained with the raw evidence.

1 Successful Jenkins execution

The local Freestyle job invokes ops/local-ci.py as a Windows batch build. It executes tests, the frontend production build, the blocking Trivy dependency scan and a real database smoke test. It is separate from the cloud Jenkinsfile and does not simulate an EKS deployment.

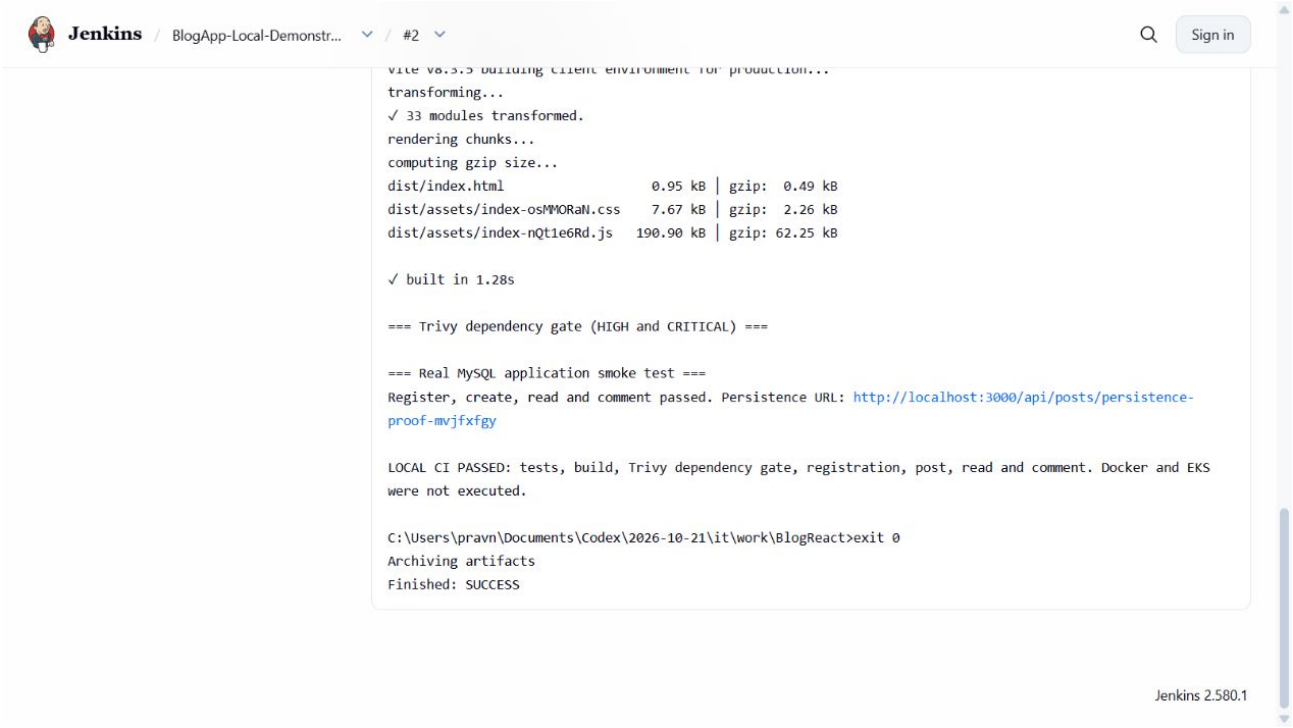


L02 - Genuine Jenkins build #2 status and archived LCOV / Trivy artifacts. URL: http://localhost:18080/job/BlogApp-Local-Demonstration/2/

Build field	Recorded value
Result	SUCCESS
Build number	2
Duration	34.448 seconds
Artifacts	backend/coverage/lcov.info and reports/trivy-local.json

Backend unit tests use a mocked database. The later smoke stage separately registers an account, creates and retrieves a post, and adds/reads a comment against the real MySQL instance. These two kinds of evidence are not interchangeable.

2 Build console and test results



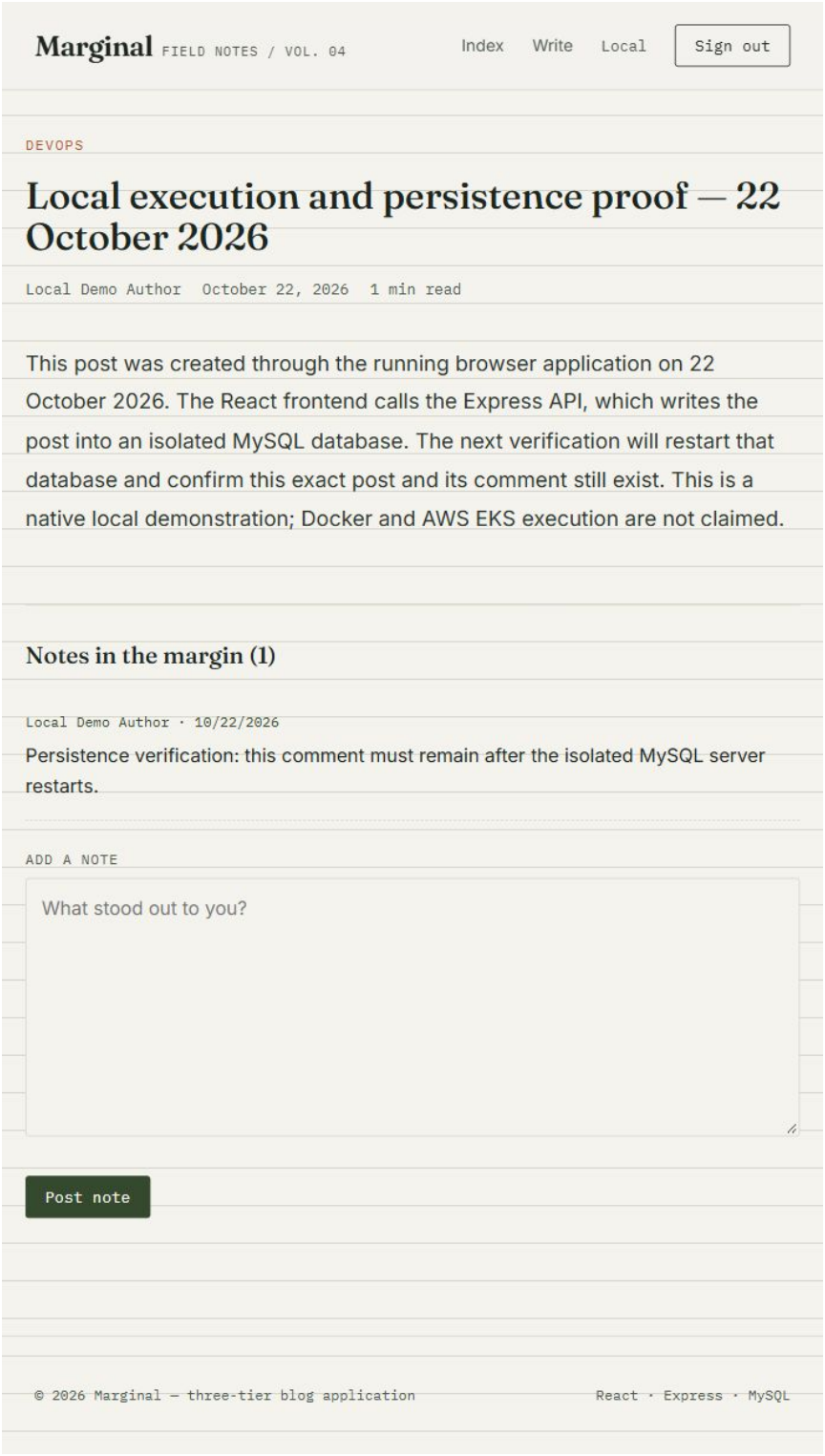
L03 - The actual console ends with the Trivy gate, real MySQL smoke operations and Finished: SUCCESS.

Stage	Actual observation
Backend tests	7 passed, 0 failed; 57.45% line coverage in loaded source files
Frontend tests	3 passed, 0 failed
Frontend build	Vite 8.3.5 transformed 33 modules and wrote production assets
Trivy gate	HIGH/CRITICAL exit-code 1 enforcement; successful scan allowed the next stage
Smoke operations	Registration, post creation, read and comment all passed

The complete unabridged console is included as jenkins-console.txt. Coverage excludes the mocked database implementation and is not a claim of complete integration coverage. There was no Git-triggered cloud release in this demonstration.

3 Browser application before restart

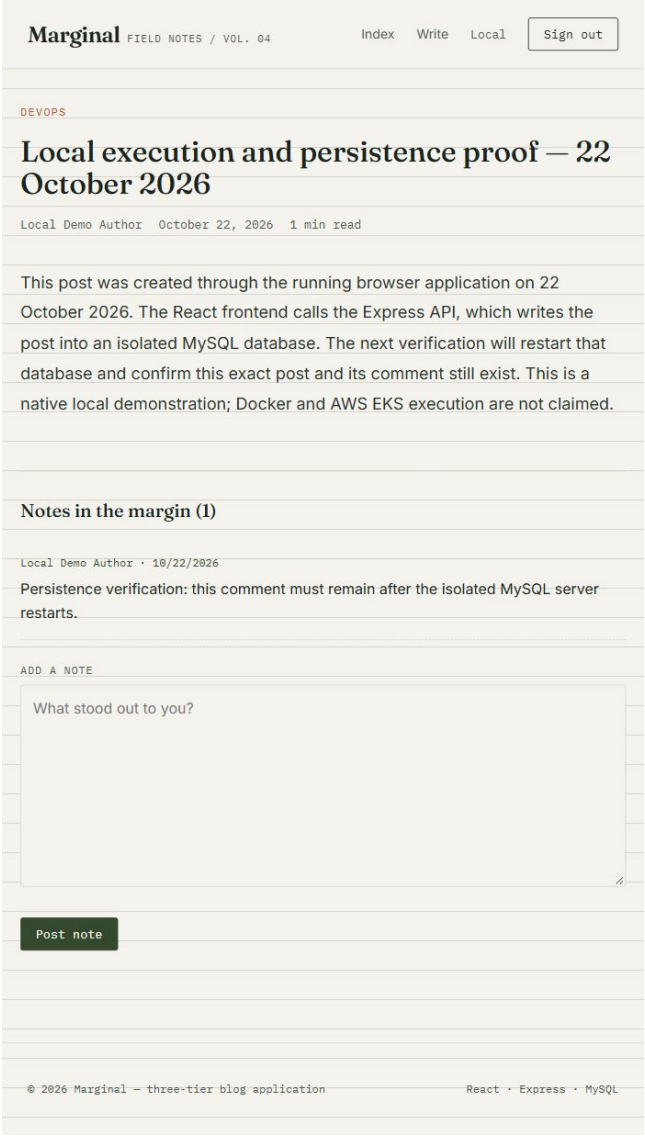
A disposable account registered through the real browser form. The author then created this post and added a comment. The browser used the React UI, which called Express and wrote to MySQL. No sample database seed supplied this proof post.



L01 - Browser-created proof post and comment before the controlled MySQL restart. The text explicitly identifies this as a native local demonstration.

4 Persistence verified after restart

Persistence assertion	Observed result
Post identity	ID 1 before and after
Comment count	1 before and after; exact comment content matched
Database storage	Same data directory, no reinitialization or schema import on restart
Health during outage	Readiness 503; liveness 200
Recovery measurement	2.09 seconds after starting MySQL

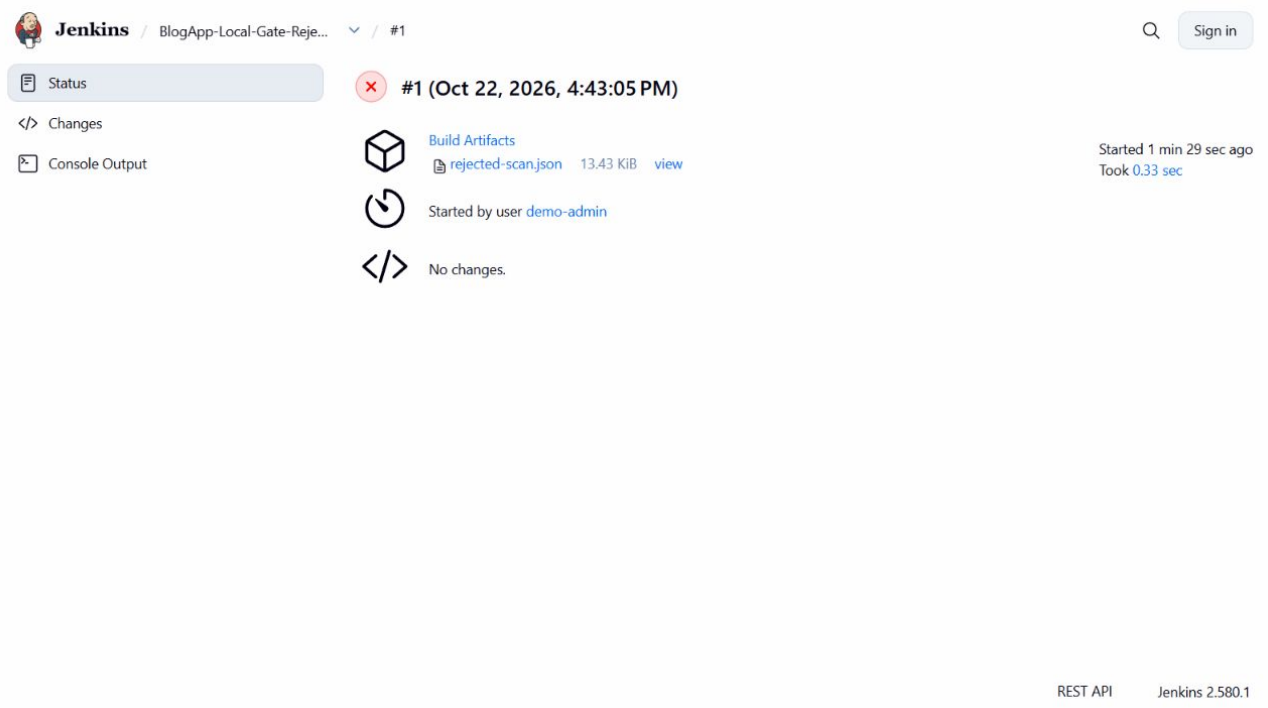


L04 - Same post and comment after database restart and browser reload. persistence-result.json records the identity/content assertions. Database-backed requests failed during the controlled outage; zero downtime is not claimed.

5 Security gate success and rejection

The successful build scanned both npm lockfiles with development dependencies included. The filter was HIGH,CRITICAL and the configured failure exit code was 1. There were zero findings at those severities. This proves a dependency gate, not an operating-system/container image scan.

Scan target	HIGH / CRITICAL findings
backend/package-lock.json	0
frontend/package-lock.json	0
Isolated lodash 4.17.20 fixture	2 HIGH; expected failure



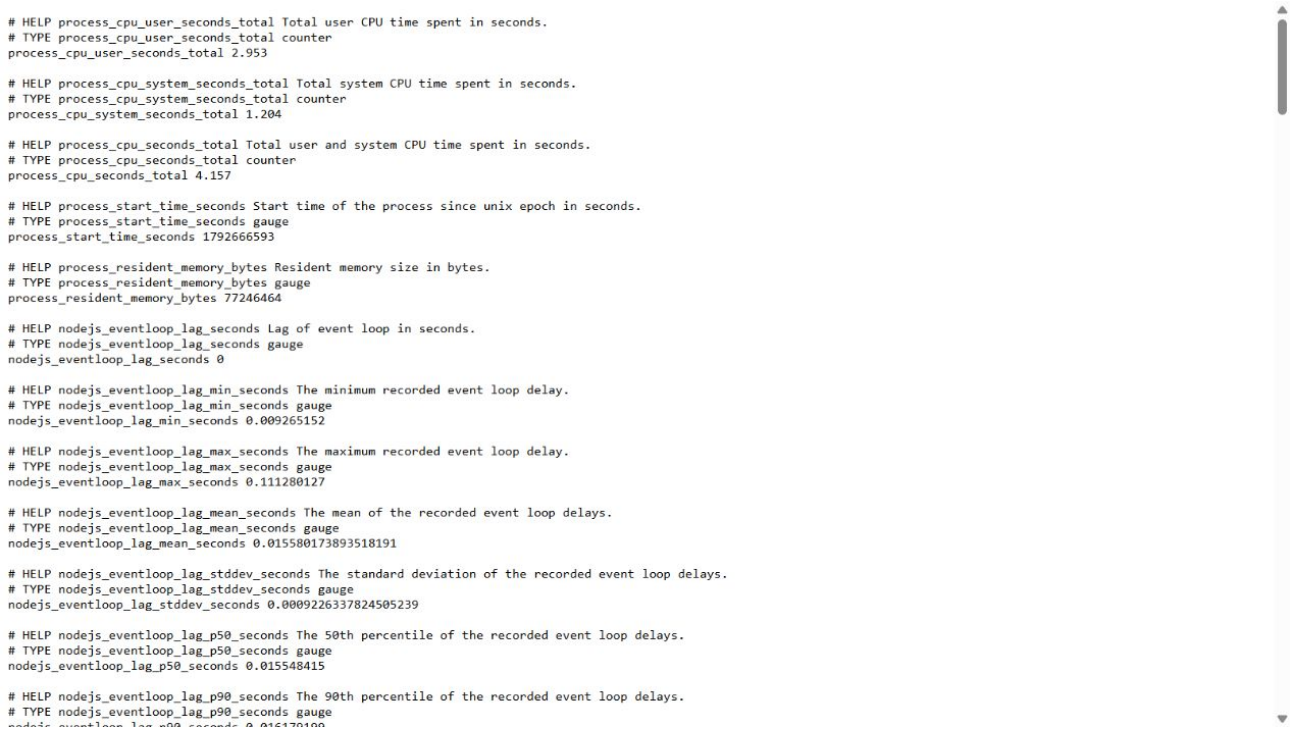
L06 - Separate Jenkins negative-test build #1 is Failed as expected. Its archived rejected-scan.json contains the findings. No vulnerable package was installed or executed.

The fixture scan reported CVE-2021-23337 and CVE-2026-4800 for lodash 4.17.20. The local negative job stopped with a nonzero scan status. It did not exercise the cloud publish/deploy stages or a SonarQube quality gate.

6 Backup restoration and live metrics

A consistent mysqldump backup was imported into a separate blogdb_restore database. The live application database was not overwritten. The dump remains private locally because it contains account records; the public evidence contains only verification results.

Restore check	Observed result
Source users / posts / comments	2 / 2 / 2 at the time of this capture
Restored users / posts / comments	2 / 2 / 2
Content comparison	Proof post ID 1 title and body matched
Backup plus restore observation	0.91 seconds; a small local test, not a production recovery objective



```
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 2.953

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 1.204

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 4.157

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1792666593

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 77246464

# HELP nodejs_eventloop_lag_seconds Lag of event loop in seconds.
# TYPE nodejs_eventloop_lag_seconds gauge
nodejs_eventloop_lag_seconds 0

# HELP nodejs_eventloop_lag_min_seconds The minimum recorded event loop delay.
# TYPE nodejs_eventloop_lag_min_seconds gauge
nodejs_eventloop_lag_min_seconds 0.009265152

# HELP nodejs_eventloop_lag_max_seconds The maximum recorded event loop delay.
# TYPE nodejs_eventloop_lag_max_seconds gauge
nodejs_eventloop_lag_max_seconds 0.111280127

# HELP nodejs_eventloop_lag_mean_seconds The mean of the recorded event loop delays.
# TYPE nodejs_eventloop_lag_mean_seconds gauge
nodejs_eventloop_lag_mean_seconds 0.015580173893518191

# HELP nodejs_eventloop_lag_stddev_seconds The standard deviation of the recorded event loop delays.
# TYPE nodejs_eventloop_lag_stddev_seconds gauge
nodejs_eventloop_lag_stddev_seconds 0.0009226337824505239

# HELP nodejs_eventloop_lag_p50_seconds The 50th percentile of the recorded event loop delays.
# TYPE nodejs_eventloop_lag_p50_seconds gauge
nodejs_eventloop_lag_p50_seconds 0.015548415

# HELP nodejs_eventloop_lag_p90_seconds The 90th percentile of the recorded event loop delays.
# TYPE nodejs_eventloop_lag_p90_seconds gauge
```

L08 - Actual Express /metrics endpoint exposes process CPU, memory and Node.js metrics in Prometheus format. Raw metrics are retained. This is not a Prometheus scrape target, Grafana dashboard, alert test or Kubernetes node metric demonstration.

The /readyz endpoint also returned {"status":"ready"} after recovery. Backend logs and the metrics include failed database requests during the controlled restart. Liveness remained available while MySQL was down.

7 Environment, evidence and limitations

Component	Actual local execution
Frontend	Node 24.19.0, React/Vite; localhost:3000; existing /api proxy
API	Express on port 5000; database port configured as 13306
Database	MySQL 8.0.42; loopback-only isolated data directory
CI	Jenkins 2.580.1 WAR with Java 25; loopback port 18080
Scanner	Trivy 0.76.1 Windows executable; official archive checksum verified

Evidence files

L01/L04 show the same application post before and after restart; L02/L03 show successful Jenkins status and console; L05 shows the running index; L06 shows expected gate rejection; L07 shows readiness; L08 shows live metrics. The accompanying ZIP includes these screenshots and raw JSON/text records, plus source hashes.

Reviewer links

Repository: https://github.com/praveenpreni/BlogReact

Report in repository: docs/Local_Execution_Report.pdf

Screenshots: docs/execution-evidence/ | Raw records: verification/local-execution/

Local URLs are available only on the demonstration computer while processes are running. They are not public application URLs. The committed PDF and screenshots provide offline review evidence.

Original brief items still unexecuted

Docker image builds and image Trivy scans; cloud Jenkins pipeline and SonarQube gate; Nexus/ECR publication; EKS workload readiness; PVC/EBS provisioning; RBAC/NetworkPolicy enforcement; rolling-update traffic; Prometheus/Grafana and alert demonstrations. The supplied implementation files support future execution, but their existence is not execution evidence.

The incomplete AWS environment was deleted at the owner's request before this local run. No cloud success or continuing cloud availability is claimed.

Reproduction and primary reference

See docs/local-demo.md and ops/local-ci.py for configuration and reproduction. Native Jenkins WAR installation: https://www.jenkins.io/doc/book/installing/war-file/